Think Twice, Act Once: Verifier-Guided Action Selection For Embodied Agents

Nishad Singhi Christian Bialas Snehal Jauhri Vignesh Prasad
Georgia Chalvatzaki Marcus Rohrbach Anna Rohrbach

Technical University of Darmstadt & hessian.AI

nishad.singhi@tu-darmstadt.de

Abstract

Building generalist embodied agents capable of solving complex real-world tasks remains a fundamental challenge in AI. Multimodal Large Language Models (MLLMs) have significantly advanced the reasoning capabilities of such agents through strong vision-language knowledge and chain-of-thought (CoT) reasoning, yet remain brittle when faced with challenging out-of-distribution scenarios. To address this, we propose ***Verifier-Guided Action Selection*** *(*VEGAS*), a test-time framework designed to improve the robustness of MLLM-based embodied agents through an explicit verification step. At inference time, rather than committing to a single decoded action,* VEGAS *samples an ensemble of candidate actions and uses a generative verifier to identify the most reliable choice, without modifying the underlying policy. Crucially, we find that using an MLLM off-the-shelf as a verifier yields no improvement, motivating our LLM-driven data synthesis strategy, which automatically constructs a diverse curriculum of failure cases to expose the verifier to a rich distribution of potential errors at training time. Across embodied reasoning benchmarks spanning the Habitat and ALFRED environments,* VEGAS *consistently improves generalization, achieving up to a* ***36% relative performance gain*** *over strong CoT baselines on the most challenging multi-object, long-horizon tasks. Code is available at* https://github.com/nishadsinghi/vegas.

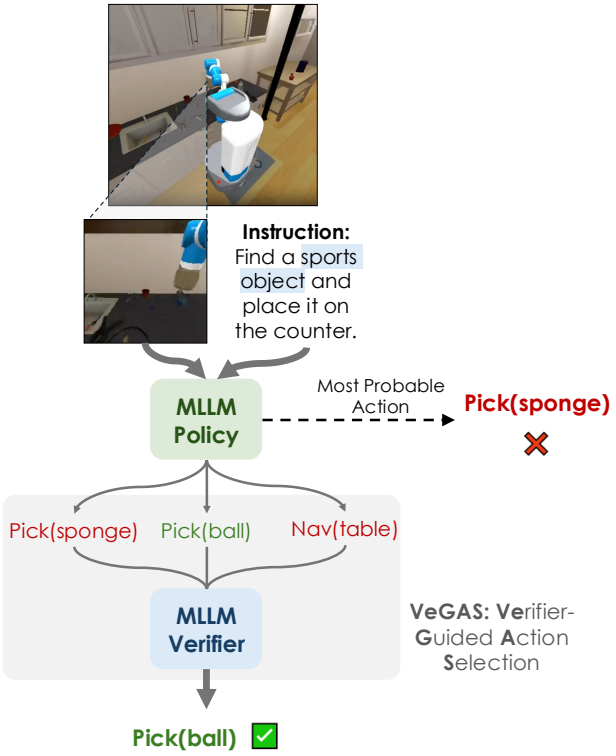


Figure 1. Overview of Verifier-Guided Action Selection (VEGAS). Given a task instruction (e.g., "Find a sports object and place it on the counter"), standard policies decode a single action that may be incorrect under distribution shifts (right). VEGAS, instead, samples multiple candidate actions with reasoning traces, evaluates them using a generative verifier, and executes only the highest-scored action (bottom). This test-time verification strategy substantially improves robustness in challenging out-of-distribution scenarios involving long-horizon tasks.

1. Introduction

A longstanding goal in AI is to create embodied agents that operate autonomously in physical environments to accomplish complex tasks specified through natural language [8, 21], such as navigating to target locations [30] and manipulating everyday objects [35, 38]. Recently, Multimodal Large Language Models (MLLMs), pretrained on Internet-scale vision-language data, have emerged as a promising foundation for building such agents, owing to their strong perceptual and linguistic generalization [14, 43, 49]. While early efforts relied on the zero-shot capabilities of MLLMs [1, 14, 31], finetuning on embodied data—either via supervised learning [50] or reinforcement learning [43, 44, 53]—yields significant improvements. More re-

cently, incorporating Chain-of-Thought (CoT) reasoning has further enhanced decision-making by enabling agents to reason step-by-step before acting [27, 47, 52, 53]. Despite this progress, MLLM-based embodied agents remain brittle in out-of-distribution scenarios and long-horizon settings [49]. For instance, an agent might reliably execute "bring me a banana" but fail when the same goal is phrased as "bring me a yellow curved fruit." Similarly, an agent trained on single-object pick-and-place may fail on a multi-step task such as cleaning an apple and placing it in a cabinet.

We observe that a key factor underlying these failures is that agents cannot recognize mistakes in their reasoning process and correct them at test time. In particular, they commit to a single greedily decoded action at each step with no opportunity for self-correction. In contrast, humans routinely consider multiple candidate actions, mentally evaluate their likely outcomes, and commit only to the most promising one, effectively performing verification before acting. This idea has a direct computational analogue: recent work on scaling test-time compute shows that sampling multiple candidate solutions and selecting the best one via a learned verifier substantially improves LLM performance in domains such as coding and mathematics [4, 5, 37]. However, extending verification to high-level embodied reasoning poses distinct challenges: unlike in mathematics or code, embodied agents operate under partial observability and must reason about semantic task progression from egocentric observations alone, with compounding errors in long-horizon plans. Yet verification for high-level embodied reasoning remains largely unexplored.

To bridge this gap, we introduce **Verifier-Guided Action Selection (VEGAS)**, a framework that improves the robustness of MLLM-based embodied agents by incorporating an explicit verification step at test time. Concretely, at each timestep VEGAS samples multiple candidate actions from the policy, each accompanied by a Chain-of-Thought rationale. A learned generative verifier [2, 54] then evaluates each candidate by producing an explicit reasoning trace followed by a correctness judgement, and the agent executes only the highest-scoring action (see Figure 1 for an overview). A critical finding is that using an MLLM as a verifier off-the-shelf yields no improvement over the base policy — general-purpose language understanding alone is insufficient for embodied verification. This motivates specialized verifier training; however, standard embodied datasets contain only successful demonstrations, providing no signal for what constitutes an incorrect action. To address this, we introduce an LLM-driven pipeline that automatically synthesizes diverse, realistic failure trajectories paired with verification annotations, constructing a rich curriculum of both correct and incorrect examples without additional human data collection.

VEGAS yields consistent improvements across embodied reasoning benchmarks in Habitat 2.0 [42, 43] and AI2-THOR [33], raising average success rates from 65% to 71% on LangR and from 44% to 49% on EB-ALFRED over strong CoT baselines, while also improving significantly larger off-the-shelf policies.

Our key contributions are:

1. We propose Verifier-Guided Action Selection (VEGAS), a test-time verification framework for high-level embodied reasoning that samples diverse candidate actions and uses a learned generative verifier to select the most reliable one at each timestep. We find that using MLLMs off-the-shelf as verifiers does not improve performance, motivating our specialized training pipeline.
2. Training a verifier requires demonstrations of both correct and incorrect actions, yet embodied datasets typically lack the latter. To address this, we introduce an automated, LLM-driven pipeline that synthesizes diverse and realistic failure trajectories paired with verification annotations, without additional human data collection.
3. Extensive experiments in Habitat 2.0 [42, 43] and AI2-THOR [33] show that VEGAS consistently improves over strong CoT baselines, scales more effectively with test-time compute than Self-Consistency [46], and generalizes to large, off-the-shelf policies.

2. Related Work

Foundation Models for Embodied Agents. Multimodal LLMs have proven useful for developing intelligent systems that perceive and interact with an environment [10, 19, 49]. They have shown strong generalization skills in areas such as Language-guided Navigation [9, 23, 30, 32], Task Planning [14, 35, 38, 39], and Embodied Decision Making [11, 20, 31, 43, 48]. While such approaches have mostly used off-the-shelf or fine-tuned LLMs/VLMs, recent works show that Chain-of-Thought (CoT) [47] can further improve performance via multi-modal reasoning [27, 52], sub-goal consistency [25], or spatial reasoning [41]. Some works have explored improving multi-agent embodied cooperation through coordinated planning [24] and tree-search-based collaborative deliberation [56]; while these methods orchestrate multiple off-the-shelf LLM agents via structured communication and joint plan search, our work targets single-agent action reliability through a dedicated verifier trained on synthetically generated failure data.

Verifiers. Verification has recently emerged as a key strategy for improving LLM reasoning by evaluating and selecting among candidate solutions. Early work trained separate verifiers to score solutions between 0 and 1, selecting the solution with the highest score as the final answer (i.e., Best-of-N) [5, 51]. Recent work has shown the advantages of generative verifiers that produce verification rationales (i.e., critiques/corrections), consistently outperforming discrimina-

tive verifiers while also enhancing explainability [2, 36, 54]. In multimodal settings, vision-language reward models extend verification to visual outcomes [40, 54], and discriminative verifiers have been applied to low-level control via VLA models [17]. In contrast, our work is the first to apply generative verifiers to high-level embodied reasoning, with an emphasis on challenging scenarios requiring novel behaviors and robustness to linguistic variations.

Embodied Agent Benchmarks. Several simulation platforms support embodied AI research, including AI2-THOR [6, 7, 12, 16] and Habitat [26, 29, 42], with benchmarks spanning diverse task complexities [15, 18, 28, 33, 34]. LangR [43], built on Habitat 2.0 [42], evaluates out-of-distribution generalization through two axes: *paraphrastic robustness* (e.g., "pick up a banana" $\rightarrow$ "pick up a yellow curved fruit") and *behavioral generalization* (e.g., extending single-object tasks to multi-object variants). ALFRED [33] contains 25K language-annotated household tasks with both high-level goals and low-level instructions across six core task types including pick-and-place, clean-and-place, and examine-in-light scenarios. TEACH [28] extends this with over 3,000 human-human dialogues for interactive task completion ranging from "Make Coffee" to "Prepare Breakfast". More recently, EmbodiedBench [49] offers a comprehensive evaluation framework with 1,128 tasks across hierarchical action levels, from high-level planning to low-level motor control, assessing capabilities such as spatial awareness and long-horizon planning.

3. Preliminaries

We now formalize the embodied decision-making setup and the policy architecture that serves as the foundation for our approach.

Problem Formulation. We formulate the agent's task as a sequential decision-making problem under partial observability. The agent's objective is to generate a sequence of actions $a_1, \ldots, a_T$ to accomplish a goal specified as a natural language instruction I (e.g., "Bring an item that can be used for cutting to the left counter"). At each timestep t, the agent receives an egocentric RGB image o_t as its observation. The agent's true underlying state s_t is not directly accessible. The agent must decide on its next action a_t based on the goal I and its history h_t composed of all its past observations and actions $(o_1, a_1, ..., o_{t-1}, a_{t-1}, o_t)$. Our aim is to learn a policy π that maps the goal and history to the next action: $\pi(a_t|I, o_{1:t}, a_{1:t-1})$. The action space A consists of high-level semantic actions, such as `pick(apple)` and `navigate(table)`. Following prior work [43, 49], we assume an oracle low-level policy that executes these high-level actions once selected.

Policy Architecture. We instantiate the policy π as a multimodal large language model (MLLM) that takes visual and text tokens as input and autoregressively generates text tokens as output. Given the goal in the form of a text instruction I and the history h_t as input, the policy autoregressively emits an output token sequence $y_t = (c_t, a_t)$. Here, c_t is an optional chain-of-thought rationale (a possibly empty sequence of text tokens) followed by the action token sequence a_t. Following Szot et al. [44], the actions are encoded in natural language (e.g., "`pick(apple)`"), and the output can be extracted to obtain the action a_t, which is sent to the environment to be executed by the low-level policy [43, 49].

Policy Training. We train the policy via imitation learning on expert demonstrations $\mathcal{D} = \{\tau\}$, where each trajectory $\tau = \big(I, (o_1, a_1), \ldots, (o_T, a_T)\big)$ depicts a successful execution of the task. The model is fine-tuned via supervised next-token prediction to maximize the likelihood of the expert output $y_t = (c_t, a_t)$, computing the loss only over output tokens, including the CoT prefix c_t when present.

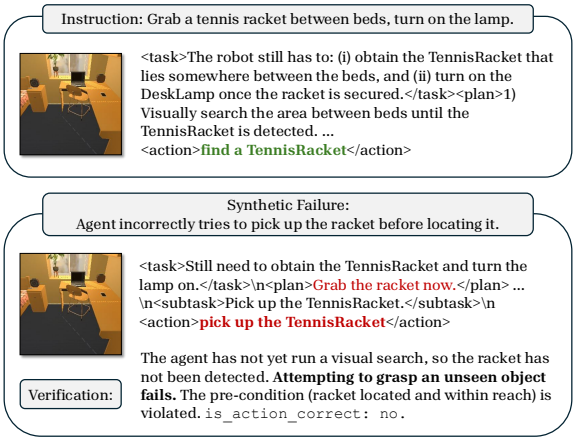


Figure 2. Example of a synthetic mistake and verification generated using our pipeline on the ALFRED training dataset. Starting from a correct action (top; 'find a TennisRacket'), our method introduces a mistake (bottom) where the agent does not locate the racket before attempting to pick it. Our method also generates a corresponding verification explaining the mistake.

4. VEGAS: Verifier-Guided Action Selection

The core idea of VEGAS is to augment a base policy with a learned verifier that, at each timestep, evaluates candidate actions and identifies the most reliable one before execution. Because off-the-shelf MLLMs fail as verifiers (as we will show in Sec. 6.1), we train a dedicated generative verifier on automatically synthesized failure data (Sec. 4.1). Concretely, VEGAS operates via a Best-of-N procedure: at each timestep, the policy samples N candidate actions, the generative verifier [2, 54] evaluates each one by producing a verification reasoning trace followed by a correctness judgement, and the highest-scoring action is executed. Unlike discriminative verifiers that directly output a score [5, 51],

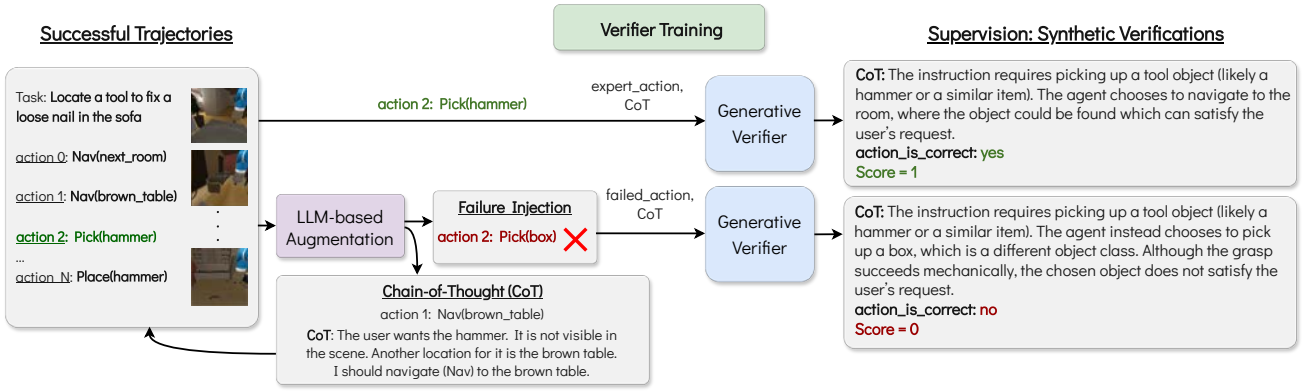


Figure 3. Synthetic data generation and training workflow for the verifier. Successful trajectories are first processed by an LLM to produce chain-of-thought rationales for each action. Then, an LLM introduces realistic and diverse errors into these trajectories and annotates every action with a verification. This dataset is used to train the verifier through supervised finetuning.

generative verifiers think step-by-step before assigning a score, which has been shown to yield stronger performance while also making the scores more interpretable [54].

4.1. Synthetic Reasoning and Verification Data

CoT Augmentation. We start with a dataset of successful ('+') trajectories, $\mathcal{D}^+ = \{\tau^+\}$, where a trajectory τ^+ consists of the instruction I, and interleaved observations o and actions a, $\tau^+ = \{I, o_1, a_1^+, o_2, a_2^+, ...\}$. A model trained on these trajectories will directly output the next action given the observation. However, research in language reasoning and embodied AI has shown that thinking step-by-step can significantly improve the reasoning abilities of models [27, 47]. To train embodied agents that can think step-by-step, similarly to Zawalski et al. [52], we prompt a teacher LLM (e.g., OpenAI o3) to augment every action with a chain-of-thought reasoning, c_i^+, explaining why the agent should perform the expected action a_i^+ given the previous inputs $I, o_1, a_1^+, ..., o_i$. This gives us a new dataset $\mathcal{D}^+_{CoT} = \{\tau^+_{CoT}\}$, with $\tau^+_{CoT} = \{I, o_1, (c_1^+, a_1^+), o_2, (c_2^+, a_2^+), ...\}$. Note that this procedure only augments every action a_i^+ with a chain-of-thought, and does not change the sequence of actions in the trajectories. Unlike Zawalski et al. [52], which grounds reasoning traces in visual features such as object and gripper positions for fine-grained manipulation, we target high-level semantic reasoning for tasks requiring long-horizon planning and linguistic interpretation, yielding $\mathcal{D}^+_{CoT}$ (Prompt in Appendix 10.1).

Synthetic Failures for Verifier Training. To train a verifier, we require examples of both correct and incorrect actions. Because existing datasets rarely include failed executions, we introduce an automated and scalable pipeline that synthesizes unsuccessful trajectories. For each successful trajectory τ^+, we prompt a large language model (e.g., OpenAI o3) to produce a corresponding failed trajectory τ^-. The model generates realistic and diverse mistakes that span a broad range of failure modes in challenging scenarios, including *wrong object* (e.g., bringing an apple when the task requires a banana), *wrong receptacle* (e.g., placing an item on the sofa instead of the bed), and *precondition violation* (e.g., attempting to turn on a microwave without opening it first). We provide examples of synthetically generated incorrect actions in Figures 2 and 3 and Appendix 11. The exact prompts are available in Appendix 10.2.

For both τ^+ and τ^-, we prompt the model to annotate every action with a verification consisting of chain-of-thought reasoning and a final binary judgement of the form `action_is_correct: yes/no`. These annotated positive and negative samples provide the supervision used to train the verifier.

4.2. Verifier Training and Inference

We fine-tune an MLLM as a verifier that takes as inputs the instruction I, all previous actions $a_1, a_2, ..., a_{t-1}$, and the current observation o_t, chain-of-thought c_t, and action a_t sampled from the policy. It outputs a verification v_t consisting of a chain-of-thought and a verdict. The verifier is trained via supervised finetuning on the data described in Sec. 4.1 using the same next-token prediction objective as the policy (Sec. 3). This process is illustrated in Figure 3.

During inference, at time t, we sample N candidate actions from the policy: $(c_t^{(1)}, a_t^{(1)}), (c_t^{(2)}, a_t^{(2)}), ..., (c_t^{(N)}, a_t^{(N)})$. We pass each candidate $(c_t^{(n)}, a_t^{(n)})$ to the verifier and, following the original GenRM procedure [54], sample M verifications per action to reduce variance, each consisting of a verification chain-of-thought and a verdict. The verdict can be mapped to a score ('`yes`' $\rightarrow$ 1 and '`no`' $\rightarrow$ 0), giving us M scores per action. We average these scores to obtain a final score for every action, $\sigma_t^{(n)}$. Finally, we select the highest-scoring action (Best-of-N): $a_t = \text{argmax}_{n \in [N]}[\sigma_t^{(n)}]$. The selected

action a_t is then executed in the environment, and the process repeats at the next timestep.

5. Experimental Setup

5.1. Benchmark Details

We evaluate our approach on two embodied AI benchmarks targeting out-of-distribution generalization: LangR [43] in the Habitat 2.0 simulator [42], and ALFRED [33] in the AI2-THOR simulator [16]. In both benchmarks, the agent is placed in previously unseen indoor environments and tasked with completing multi-step household tasks (e.g., rearranging objects, examining items) specified through natural language. At each timestep, the agent receives an egocentric RGB observation and selects a high-level semantic action (e.g., `navigate(table)`, `pick(apple)`, `open(fridge)`), which is executed by the simulator.

LangR [43]. This benchmark comprises a diverse set of training tasks featuring multiple paraphrastic variations and interactions with different household objects. The benchmark also includes several out-of-distribution (OOD) tasks designed to evaluate the model's generalization capabilities. These evaluation tasks differ from the training set in their natural language instructions, which vary either through linguistic reformulation (termed *Paraphrastic Robustness*, e.g., "pick up a banana" $\rightarrow$ "pick up a yellow curved fruit") or through changes in the underlying task structure (termed *Behavioral Generalization*, e.g., "move an apple and a banana" $\rightarrow$ "move an apple, a banana, and a ball"). The evaluation suite comprises 8 tasks with 100 instructions each. Further details are available in Appendix 8.

ALFRED [33]. This benchmark, built upon the AI2-THOR simulator [16], encompasses seven distinct task types (such as *pick and place* and *examine in light*) that involve diverse interactions with objects in household environments. In this work, we employ the EB-ALFRED implementation introduced by Yang et al. [49], which reorganizes tasks from the original benchmark into several categories designed to assess different aspects of OOD generalization, including *long-horizon tasks*, *common-sense reasoning*, and *spatial understanding*. The evaluation suite comprises 6 tasks with 50 instructions each. Further details are in Appendix 8.

5.2. Policy and Verifier

Policy Training. LangR does not include expert demonstrations for training. To construct its training dataset $\mathcal{D}^+$, we execute an RL-trained policy from Szot et al. [43] on the LangR training split, collecting 10K trajectories, each comprising an instruction, observations, and actions. Trajectories in which the agent failed to complete the task are discarded (fewer than 3% of the total). For EB-ALFRED, we use the instructions, observations, and actions from the training data provided by the original ALFRED benchmark [33], which contains approximately 6.5K expert demonstrations. For both benchmarks, we prompt OpenAI's o3 model to augment every action in the expert trajectories with a chain-of-thought (see prompts in Appendix 10.1), yielding $\mathcal{D}^+_{CoT}$ (as described in Sec. 4.1). We fine-tune Qwen2.5-VL-3B-Instruct [3] on $\mathcal{D}^+_{CoT}$ to obtain the chain-of-thought (CoT) policy. Additional implementation and hyperparameter details are provided in Appendix 9.

Verifier Training. We begin with approximately 4.5K and 6.5K successful trajectories from LangR and ALFRED, respectively. To generate negative samples, we prompt OpenAI's o3 model to synthesize one failed trajectory corresponding to each successful example, and to provide verification annotations for every action within the failed trajectory (see prompts in Appendix 10.2). This process yields a corpus of failed trajectories, denoted as $\mathcal{D}^-_{CoT}$. The same model is further used to annotate each action in the successful trajectories with corresponding verifications. We combine the verifications from both successful and failed trajectories, and randomly sample from this pool to construct a balanced dataset containing equal numbers of correct and incorrect samples. To ensure a fair comparison, we use the same base model—Qwen2.5-VL-3B-Instruct [3]—for both the policy and the verifier, differing only in their training data. Additional implementation details are provided in Appendix 9.

Inference. We use `vLLM` [22] to perform inference with the policy and verifier. For Habitat 2.0, we run experiments on NVIDIA L40 GPUs. For ALFRED, we perform experiments on NVIDIA A100 80GB GPUs. For the No-CoT and CoT policies, we sample the model responses via greedy decoding. When sampling multiple candidate actions, we sample actions and verifications with a temperature of 0.7. For VEGAS, we sample $N = 16$ candidate actions and $M = 5$ verifications per action at every timestep. We report results and comparisons against baselines in Sec. 6.

6. Experiments

First, we evaluate the impact of zero-shot and finetuned verifiers on out-of-distribution generalization (Sec. 6.1). Next, we examine whether our finetuned verifier can improve larger, off-the-shelf policies it was never trained with (Sec. 6.1). Finally, we present ablation studies examining training pipeline choices, test-time compute scaling, and latency (Sec. 6.2).

6.1. Verifiers Improve Generalization, But Only When Finetuned

We analyze the effect of verification on the LangR benchmark (Table 1). Fine-tuning with CoT supervision achieves 65% average success rate, surpassing prior state-of-the-art SemLang [44] and establishing a strong baseline.

We then evaluate whether verification can further improve the CoT policy. Using the same Qwen2.5-VL-3B-

Approach	Average	Paraphrastic Robustness				Behavioral Generalization			
		Rephrasing	Context	Irrelevant Text	Referring Expressions	Multiple Rearrange	Novel Objects	Multiple Objects	Conditional
Prior Work									
LLaRP (LLaMa-7B) [43]	46 ± 2	92 ± 2	34 ± 2	32 ± 2	26 ± 2	47 ± 5	95 ± 4	0 ± 1	39 ± 3
SemLang (LLaVA-1.5-7B) [44]	58 ± 1	92 ± 1	46 ± 14	66 ± 6	31 ± 3	80 ± 6	97 ± 0	2 ± 2	46 ± 4
Policy only (Qwen-2.5-VL-3B-Instruct)									
No-CoT	58	93	39	72	48	68	97	17	28
w/ CoT	65	98	50	85	59	64	97	25	42
w/ CoT policy + Verifier (Qwen-2.5-VL-3B-Instruct)									
+ ZS Verifier	64 ± 2	98 ± 1	50 ± 3	85 ± 5	48 ± 4	65 ± 4	97 ± 0	30 ± 4	40 ± 3
+ FT Verifier (VEGAS)	**71 ± 1**	**99 ± 1**	**52 ± 2**	**92 ± 1**	**62 ± 3**	**82 ± 1**	**97 ± 0**	**34 ± 2**	**48 ± 2**

Table 1. Success rates on the LangR [43] benchmark. The No-CoT, w/ CoT, and VEGAS models are based on Qwen-2.5-VL-3B-Instruct. ZS and FT refer to zero-shot and finetuning, respectively. For verifier-based approaches, results are averaged over three runs. The No-CoT and CoT variants use greedy decoding for action selection, yielding deterministic outcomes. Our proposed approach, which combines chain-of-thought reasoning with a finetuned verifier (VEGAS), consistently outperforms all baselines.

Instruct model as a zero-shot verifier (+ ZS Verifier) does not meaningfully improve over the CoT baseline and, in fact, slightly hurts average performance (64% vs. 65%). This shows that the Best-of-N selection paradigm alone is insufficient; without task-specific training, the verifier cannot reliably distinguish correct actions from incorrect ones. In contrast, equipping the CoT policy with our finetuned verifier (VEGAS) raises performance to 71%, with consistent gains across all task categories. The improvements are particularly pronounced in challenging scenarios such as *Multiple Objects*, where VEGAS provides roughly a 36% relative improvement over CoT alone and doubles performance compared to No-CoT. These results demonstrate that the gains of VEGAS stem not from sampling multiple candidates per se, but from our synthetic failure generation and verifier training pipeline. An example in Figure 4 highlights the verifier's effectiveness, where it correctly flags an action arising from the agent misunderstanding the instruction.

To test whether these findings generalize beyond LangR, we repeat the evaluation on EB-ALFRED (Table 2). Our CoT policy, obtained by fine-tuning Qwen2.5-VL-3B-Instruct, achieves an average success rate of 44%, as shown in Table 2. This surpasses Qwen2.5-VL-72B-Instruct (success rate 30%), a $\sim$20$\times$ larger model, and the best open-weights model reported on EB-ALFRED [49], thereby establishing a strong baseline. As on LangR, using the same model as a zero-shot verifier (+ ZS Verifier) does not improve over the CoT baseline (44% vs. 44%). In contrast, equipping the policy with our finetuned verifier (VEGAS) raises performance to 49%, confirming that task-specific verifier training is essential for effective verification. To assess whether these findings generalize beyond a single backbone, we repeat the experiment with Gemma-3-4B [45], fine-tuning both the CoT policy and the verifier. We observe the same trends: the zero-shot verifier provides no meaningful gains, while VEGAS improves the average success rate to 51%. This consistency across two different model families demonstrates that the benefits of VEGAS are not architecture-specific but arise from the finetuned verifier itself. Taken together, our results on LangR and EB-ALFRED demonstrate that verifying actions at test time can substantially enhance the generalization capabilities of embodied agents in challenging scenarios. Figure 5 shows an example where the verifier reliably detects a subtle error made by the policy.

Verifier-Guided Improvement of Large Policies. We evaluate whether a small, finetuned verifier can improve large policies it was never trained with — a practical setting where large models are inaccessible for fine-tuning. We pair our Qwen2.5-VL-3B-Instruct verifier with several zero-shot policies on EB-ALFRED (Table 2). Our verifier consistently improves every policy it is paired with; most notably, it improves Qwen-2.5-VL-72B—a model $\sim$20$\times$ its own size—from 30% to 38%, demonstrating that a compact verifier can meaningfully enhance policies far beyond its own scale.

6.2. Ablation Studies

Impact of Scaling Candidate Actions. Increasing the number of sampled candidate actions increases the likelihood of including at least one correct action [4], but also increases inference cost. To isolate whether gains come from the verifier or from sampling diversity alone, we compare against self-consistency [46], which samples multiple actions and selects by majority vote. To ensure a fair comparison, we match total LLM calls across both methods: following Singhi et al. [36], if VEGAS samples N actions with M verifications each, self-consistency samples $N(M+1)$ actions.[1] As shown

[1]The total number of LLM calls for VEGAS is $N + N \times M = N(M+1)$: N policy calls plus $N \times M$ verification calls.

Instruction: Acquire a sports object and position it in the brown table.

Sampled Action: <task>Acquire a sports object and position it on the brown table; remaining: locate the object, pick it up, navigate to the brown table, and place it.</task> <plan>[search sink, search Table1, search TV stand, search sofa, pick object, navigate to brown table, place object]</plan> <subtask_reason>Having arrived at the sink, I must check if the object is here and grasp it if present; hence I need to do pick_sponge(robot_0).</subtask_reason> <subtask>attempt pick at sink</subtask> <action>('pick_sponge(robot_0)', 28)</action>

Verification CoT: The instruction requires picking up a SPORTS object (likely a ball). The chosen action is pick_sponge, which grasps a sponge instead. It does not move the correct object toward completion of the task and therefore fails to advance the goal. **action_is_correct: no**

Figure 4. Verification example on the LangR benchmark. Here, the task is to move a "sports object", but the agent proposes 'pick_sponge'. The verifier correctly identifies that the sponge is not a sports object, classifying the action as incorrect.

Approach	Average	Base	Common Sense	Complex Instructions	Long Horizon	Spatial	Visual Appearance
Finetuned Policies							
Qwen-3B w/ CoT	44	62	40	58	22	34	**48**
+Qwen-3B ZS Verifier	44 ± 2	64 ± 2	41 ± 7	53 ± 5	24 ± 3	35 ± 4	46 ± 2
+Qwen-3B FT Verifier (VEGAS)	**49 ± 2**	**67 ± 5**	**46 ± 2**	**62 ± 3**	**34 ± 2**	**43 ± 3**	41 ± 1
Gemma-4B w/ CoT	48	62	50	56	**28**	34	**56**
+Gemma-4B ZS Verifier	48 ± 2	66 ± 0	55 ± 1	61 ± 5	27 ± 3	32 ± 4	49 ± 6
+Gemma-4B FT Verifier (VEGAS)	**51 ± 1**	**67 ± 1**	**56 ± 0**	**67 ± 1**	25 ± 1	**37 ± 1**	53 ± 6
Zero-shot Policies							
Qwen-2.5-VL-72B	30	28	38	34	12	**38**	32
+Qwen-3B FT Verifier (VEGAS)	**38 ± 1**	**45 ± 3**	**39 ± 6**	**44 ± 7**	**15 ± 3**	36 ± 3	**47 ± 1**
Gemma-3-27B	19	24	24	26	0	**24**	18
+Qwen-3B FT Verifier (VEGAS)	**23 ± 0**	**30 ± 2**	**29 ± 1**	**27 ± 1**	**2 ± 2**	23 ± 1	**25 ± 1**
InternVL-3.5-38B	24	32	26	26	10	18	30
+Qwen-3B FT Verifier (VEGAS)	**35 ± 2**	**38 ± 2**	**36 ± 5**	**39 ± 5**	**27 ± 3**	**26 ± 2**	**41 ± 3**

Table 2. Success rates on EB-ALFRED [49]. ZS and FT refer to zero-shot and finetuning, respectively. The CoT policy employs greedy decoding, producing deterministic outcomes. Results involving verifiers are averaged over three runs. Our finetuned verifier consistently outperforms the CoT policy as well as the zero-shot verifier. Further, our finetuned verifier improves performance of larger policies, showing cross-model generalization.

in Figure 6, on EB-ALFRED, self-consistency improves with additional compute but does not scale as efficiently as VEGAS, which exhibits steeper and more consistent gains. These results underscore that a finetuned verifier is crucial for effectively leveraging additional test-time compute to improve embodied agent performance.

Sampled Candidates Reliably Contain Correct Actions. Best-of-N selection can only succeed if the candidate set contains at least one correct action. We therefore ask: how often does at least one correct action appear among N sampled candidates? Since no ground-truth oracle exists for action correctness, we use o3 as a judge. We measure this coverage probability on LangR as a function of N. As shown in Table 3, the coverage rises sharply with N: with just 10 candidates, at least one correct action is present in 89% of cases. This confirms that the policy's candidate set is highly likely to contain a correct action, making Best-of-N an effective strategy.

N (candidates)	P(at least one correct)
2	0.681
4	0.794
10	0.894

Table 3. Candidate set coverage on LangR. Probability that at least one correct action appears among N candidates.

Teacher Model Sensitivity. We investigate whether replacing o3 with the cheaper Qwen-3-VL-8B-thinking as teacher degrades verifier quality. As shown in Table 4, the weaker-teacher verifier achieves 69%, compared to 71% with o3-generated data and 65% for the CoT baseline. While o3 yields the best performance, a much cheaper teacher still provides meaningful gains, making our pipeline accessible without requiring expensive frontier models.

Instruction: Move the book from the table to the large comfortable lounging spot.

Sampled Action: <task>Book is in hand; it still must be placed on the large comfortable lounge chair.</task> <plan>[1) Find/navigate to the lounge chair, 2) Put the book down]</plan> <subtask_reason>I cannot place the book until I have reached the correct furniture item. Its exact location relative to the current viewpoint is not confirmed, so I should perform another search targeted at an ArmChair, hence I should choose the action that finds an ArmChair.</subtask_reason> <subtask>Find the lounge chair</subtask> <action>find a ArmChair</action>.

Verification CoT: The instruction specifies the large comfortable lounging spot, which in the environment corresponds to the Sofa object. Searching for an ArmChair indicates the agent has mis-identified the target receptacle. While the action executes successfully, it does not advance the true goal. **action_is_correct: no**

Figure 5. Verification example on the ALFRED benchmark. The task is to move the book to a "large comfortable lounging spot", but the agent makes a subtle mistake and proposes the related receptacle, armchair. The verifier correctly identifies that the correct receptacle is Sofa, classifying the action as incorrect.

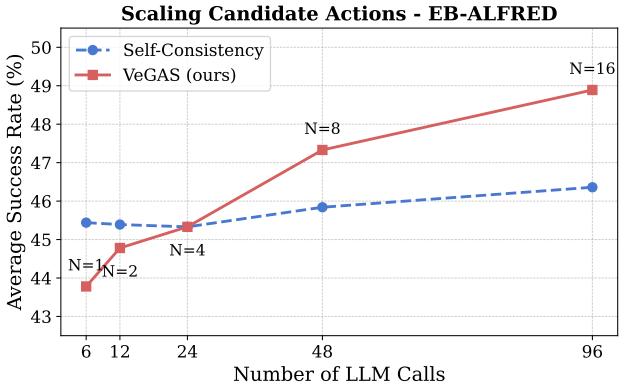


Figure 6. Scaling candidate actions on EB-ALFRED. Average success rate as the number of candidate actions N increases. Both methods use the same total number of LLM calls. VEGAS scales better with compute than Self-Consistency.

Approach	Avg. Success Rate
w/ CoT (no verifier)	65
+ FT Verifier (Qwen3-8B teacher)	69
+ FT Verifier (o3 teacher)	**71**

Table 4. Teacher model sensitivity on LangR. Average success rate when using Qwen-3-VL-8B-thinking vs. o3 to generate synthetic verifier training data.

Latency. A natural concern with VEGAS is inference latency: sampling N candidate actions and M verifications per action requires $N(M+1)$ total LLM calls. However, since all candidates and verifications can be sampled in parallel, the wall-clock overhead is far more modest than the raw call count suggests. Table 5 reports latency as we scale N at $M = 5$. Going from $N = 1$ (a single greedy action, 1 LLM call) to $N = 8$ (48 LLM calls, 48× more) increases latency by only 2× (3s → 6s). This demonstrates that parallel sampling makes VEGAS practical for deployment even at larger compute budgets.

Config	LLM Calls	Latency
$N = 1$	1	3s
$N = 4,\ M = 5$	24	5s
$N = 8,\ M = 5$	48	6s
$N = 16,\ M = 5$	96	8s

Table 5. Latency vs. compute budget. Wall-clock time to sample all actions and verifications for increasing N (candidate actions).

Impact of Visual Input to the Verifier. We ask: if we train a verifier that is text-only (i.e., receiving only the action and its reasoning CoT, without the egocentric image), how much does performance degrade? On LangR, the text-only verifier achieves the same average success rate as the multimodal verifier (71% vs. 71%). On EB-ALFRED, we observe only a marginal drop (49% vs. 47.5%). We note that the text-only verifier is not truly "blind": the chain-of-thought reasoning trace accompanying each candidate action describes the visual scene in natural language, which may explain why removing the image input does not meaningfully hurt performance. This is consistent with prior work showing that text-based scene descriptions are sufficient for high-level embodied planning [13, 49]. We also speculate that current high-level benchmarks lack complex scenarios with occlusions or fine-grained visual distinctions where explicit vision-based verification would be most beneficial.

7. Conclusion

We introduced **Verifier-Guided Action Selection (VEGAS)**, a test-time framework that improves the out-of-distribution robustness of embodied agents via an explicit verification step. Using an automated pipeline to synthesize failure trajectories for verifier training, VEGAS achieves consistent gains on LangR and EB-ALFRED, including over significantly larger off-the-shelf policies. Our analyses show that verifier finetuning is essential for reliably leveraging additional test-time compute.

Acknowledgements

Nishad Singhi is supported by a LOEWE Start-Professur (LOEWE/4b//519/05.01.002-(0006)/94). Marcus Rohrbach is supported in part by an Alexander von Humboldt Professorship in Multimodal Reliable AI, sponsored by Germany's Federal Ministry for Education and Research. For compute, we gratefully acknowledge support from the hessian.AI Service Center (funded by the Federal Ministry of Research, Technology and Space, BMFTR, grant no. 16IS22091) and the hessian.AI Innovation Lab (funded by the Hessian Ministry for Digital Strategy and Innovation, grant no. S-DIW04/0013/003). The work has benefited from the Excellence Cluster "Reasonable AI" by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) under Germany's Excellence Strategy – EXC-3057, DFG Emmy Noether Programme (CH 2676/1-1), European Union's Horizon Europe project "MANiBOT" (Grant No.: 101120823), European Union's Horizon Europe project "ARISE" (Grant No.: 101135959), BMFTR Project "RIG" (Grant No.: 16ME1001).

References

[1] Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman, et al. Do as i can, not as i say: Grounding language in robotic affordances. *arXiv preprint arXiv:2204.01691*, 2022. 1

[2] Zachary Ankner, Mansheej Paul, Brandon Cui, Jonathan D Chang, and Prithviraj Ammanabrolu. Critique-out-loud reward models. *arXiv preprint arXiv:2408.11791*, 2024. 2, 3

[3] Shuai Bai, Shusheng Yang, Shixi Wang, Changzhi Sun, Zheng Gong, Yu Yang, Yuhao Qian, Xiaoteng Ren, Zhiyang Wei, Zhuo Su, et al. Qwen2.5-vl: A state-of-the-art vision-language model series. *arXiv preprint arXiv:2502.13923*, 2025. 5

[4] Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V Le, Christopher Ré, and Azalia Mirhoseini. Large language monkeys: Scaling inference compute with repeated sampling. *arXiv preprint arXiv:2407.21787*, 2024. 2, 6

[5] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021. 2, 3

[6] Matt Deitke, Winson Han, Alvaro Herrasti, Aniruddha Kembhavi, Eric Kolve, Roozbeh Mottaghi, Jordi Salvador, Dustin Schwenk, Eli VanderBilt, Matthew Wallingford, Luca Weihs, Mark Yatskar, and Ali Farhadi. RoboTHOR: An Open Simulation-to-Real Embodied AI Platform. In *CVPR*, 2020. 3

[7] Matt Deitke, Eli VanderBilt, Alvaro Herrasti, Luca Weihs, Jordi Salvador, Kiana Ehsani, Winson Han, Eric Kolve, Ali Farhadi, Aniruddha Kembhavi, and Roozbeh Mottaghi. ProcTHOR: Large-Scale Embodied AI Using Procedural Generation. In *NeurIPS*, 2022. Outstanding Paper Award. 3

[8] Yan Ding, Xiaohan Zhang, Chris Paxton, and Shiqi Zhang. Task and motion planning with large language models for object rearrangement. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2023. 1

[9] Vishnu Sashank Dorbala, James F Mullen, and Dinesh Manocha. Can an embodied agent find your "cat-shaped mug"? llm-based zero-shot object navigation. *IEEE Robotics and Automation Letters*, 2023. 2

[10] Mengfei Du, Binhao Wu, Zejun Li, Xuan-Jing Huang, and Zhongyu Wei. Embspatial-bench: Benchmarking spatial understanding for embodied tasks with large vision-language models. In *Annual Meeting of the Association for Computational Linguistics (Short Papers)*, 2024. 2

[11] Yuqing Du, Olivia Watkins, Zihan Wang, Cédric Colas, Trevor Darrell, Pieter Abbeel, Abhishek Gupta, and Jacob Andreas. Guiding pretraining in reinforcement learning with large language models. In *International Conference on Machine Learning (ICML)*, 2023. 2

[12] Kiana Ehsani, Winson Han, Alvaro Herrasti, Eli VanderBilt, Luca Weihs, Eric Kolve, Aniruddha Kembhavi, and Roozbeh Mottaghi. ManipulaTHOR: A Framework for Visual Object Manipulation. In *CVPR*, 2021. 3

[13] Jiani Huang, Amish Sethi, Matthew Kuo, Mayank Keoliya, Neelay Velingker, JungHo Jung, Ser-Nam Lim, Ziyang Li, and Mayur Naik. Esca: Contextualizing embodied agents via scene-graph generation. *arXiv preprint arXiv:2510.15963*, 2025. 8

[14] Wenlong Huang, Pieter Abbeel, Deepak Pathak, and Igor Mordatch. Language models as zero-shot planners: Extracting actionable knowledge for embodied agents. In *International conference on machine learning*, pages 9118–9147. PMLR, 2022. 1, 2

[15] Yash Kant, Arun Ramachandran, Sriram Yenamandra, Igor Gilitschenski, Dhruv Batra, Andrew Szot, and Harsh Agrawal. Housekeep: Tidying virtual households using commonsense reasoning. In *European Conference on Computer Vision*, pages 355–373. Springer, 2022. 3

[16] Eric Kolve, Roozbeh Mottaghi, Winson Han, Eli VanderBilt, Luca Weihs, Alvaro Herrasti, Matt Deitke, Kiana Ehsani, Daniel Gordon, Yuke Zhu, et al. Ai2-thor: An interactive 3d environment for visual ai. *arXiv preprint arXiv:1712.05474*, 2017. 3, 5, 1

[17] Jacky Kwok, Christopher Agia, Rohan Sinha, Matt Foutter, Shulu Li, Ion Stoica, Azalia Mirhoseini, and Marco Pavone. Robomonkey: Scaling test-time sampling and verification for vision-language-action models. *arXiv preprint arXiv:2506.17811*, 2025. 3

[18] Chengshu Li, Ruohan Zhang, Josiah Wong, Cem Gokmen, Sanjana Srivastava, Roberto Martín-Martín, Chen Wang, Gabrael Levine, Michael Lingelbach, Jiankai Sun, et al. Behavior-1k: A benchmark for embodied ai with 1,000 everyday activities and realistic simulation. In *Conference on Robot Learning*, pages 80–93. PMLR, 2023. 3

[19] Manling Li, Shiyu Zhao, Qineng Wang, Kangrui Wang, Yu Zhou, Sanjana Srivastava, Cem Gokmen, Tony Lee, Erran Li Li, Ruohan Zhang, et al. Embodied agent interface: Benchmarking llms for embodied decision making. *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. 2

[20] Shuang Li, Xavier Puig, Chris Paxton, Yilun Du, Clinton Wang, Linxi Fan, Tao Chen, De-An Huang, Ekin Akyürek, Anima Anandkumar, et al. Pre-trained language models for interactive decision-making. *Advances in Neural Information Processing Systems*, 35:31199–31212, 2022. 2

[21] Wenhao Li, Zhiyuan Yu, Qijin She, Zhinan Yu, Yuqing Lan, Chenyang Zhu, Ruizhen Hu, and Kai Xu. Llm-enhanced scene graph learning for household rearrangement. In *SIGGRAPH Asia 2024*, 2024. 1

[22] Zhuohan Li, Eric Jin, Xiang Li, Haotian Luo, Minjia Zhang, Mohammad Shoeybi, Tianyu Du, Shouhan Wang, Jimeng Sun, Shoumeng Yan, Zeming Yu, Xin He, Yiming Wang, Michael Wiseman, Amr Ahmed, Mu Li, Ce Zhang, Joseph E. Gonzalez, Ion Stoica, and Tuo Zhao. vllm: Easy, fast, and cheap llm inference. In *Proceedings of Neural Information Processing Systems (NeurIPS) Demo Track*, 2023. 5

[23] Bingqian Lin, Yunshuang Nie, Ziming Wei, Jiaqi Chen, Shikui Ma, Jianhua Han, Hang Xu, Xiaojun Chang, and Xiaodan Liang. Navcot: Boosting llm-based vision-and-language navigation via learning disentangled reasoning. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2025. 2

[24] Jie Liu, Pan Zhou, Yingjun Du, Ah-Hwee Tan, Cees GM Snoek, Jan-Jakob Sonke, and Efstratios Gavves. Capo: Cooperative plan optimization for efficient embodied multi-agent cooperation. *arXiv preprint arXiv:2411.04679*, 2024. 2

[25] Guanxing Lu, Ziwei Wang, Changliu Liu, Jiwen Lu, and Yansong Tang. Thinkbot: Embodied instruction following with thought chain reasoning. In *International Conference on Learning Representations (ICLR)*, 2025. 2

[26] Manolis Savva*, Abhishek Kadian*, Oleksandr Maksymets*, Yili Zhao, Erik Wijmans, Bhavana Jain, Julian Straub, Jia Liu, Vladlen Koltun, Jitendra Malik, Devi Parikh, and Dhruv Batra. Habitat: A Platform for Embodied AI Research. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019. 3

[27] Yao Mu, Qinglong Zhang, Mengkang Hu, Wenhai Wang, Mingyu Ding, Jun Jin, Bin Wang, Jifeng Dai, Yu Qiao, and Ping Luo. Embodiedgpt: Vision-language pre-training via embodied chain of thought. *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. 2, 4

[28] Aishwarya Padmakumar, Jesse Thomason, Ayush Shrivastava, Patrick Lange, Anjali Narayan-Chen, Spandana Gella, Robinson Piramuthu, Gokhan Tur, and Dilek Hakkani-Tur. Teach: Task-driven embodied agents that chat. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 2017–2025, 2022. 3

[29] Xavier Puig, Eric Undersander, Andrew Szot, Mikael Dallaire Cote, Tsung-Yen Yang, Ruslan Partsey, Ruta Desai, Alexander Clegg, Michal Hlavac, So Yeon Min, et al. Habitat 3.0: A co-habitat for humans, avatars, and robots. In *International Conference on Learning Representations (ICLR)*, 2024. 3

[30] Yanyuan Qiao, Wenqi Lyu, Hui Wang, Zixu Wang, Zerui Li, Yuan Zhang, Mingkui Tan, and Qi Wu. Open-nav: Exploring zero-shot vision-and-language navigation in continuous environment with open-source llms. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2025. 1, 2

[31] Gabriel Sarch, Yue Wu, Michael Tarr, and Katerina Fragkiadaki. Open-ended instructable embodied agents with memory-augmented large language models. In *ACL Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023. 1, 2

[32] Raphael Schumann, Wanrong Zhu, Weixi Feng, Tsu-Jui Fu, Stefan Riezler, and William Yang Wang. Velma: Verbalization embodiment of llm agents for vision and language navigation in street view. In *AAAI Conference on Artificial Intelligence*, 2024. 2

[33] Mohit Shridhar, Jesse Thomason, Daniel Gordon, Yonatan Bisk, Winson Han, Roozbeh Mottaghi, Luke Zettlemoyer, and Dieter Fox. Alfred: A benchmark for interpreting grounded instructions for everyday tasks. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10740–10749, 2020. 2, 3, 5, 1

[34] Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Cote, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. Alfworld: Aligning text and embodied environments for interactive learning. In *International Conference on Learning Representations (ICLR)*, 2021. 3

[35] Ishika Singh, Valts Blukis, Arsalan Mousavian, Ankit Goyal, Danfei Xu, Jonathan Tremblay, Dieter Fox, Jesse Thomason, and Animesh Garg. Progprompt: Generating situated robot task plans using large language models. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2023. 1, 2

[36] Nishad Singhi, Hritik Bansal, Arian Hosseini, Aditya Grover, Kai-Wei Chang, Marcus Rohrbach, and Anna Rohrbach. When to solve, when to verify: Compute-optimal problem solving and generative verification for llm reasoning. In *Conference on Language Modelling (COLM)*, 2025. 3, 6

[37] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling llm test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024. 2

[38] Chan Hee Song, Jiaman Wu, Clayton Washington, Brian M Sadler, Wei-Lun Chao, and Yu Su. Llm-planner: Few-shot grounded planning for embodied agents with large language models. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023. 1, 2

[39] Haotian Sun, Yuchen Zhuang, Lingkai Kong, Bo Dai, and Chao Zhang. Adaplanner: Adaptive planning from feedback with language models. *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. 2

[40] Linzhuang Sun, Hao Liang, Jingxuan Wei, Bihui Yu, Tianpeng Li, Fan Yang, Zenan Zhou, and Wentao Zhang. Mm-verify: Enhancing multimodal reasoning with chain-of-thought verification. *arXiv preprint arXiv:2502.13383*, 2025. 3

[41] Qi Sun, Pengfei Hong, Tej Deep Pala, Vernon Toh, U-Xuan Tan, Deepanway Ghosal, and Soujanya Poria. Emma-x: An embodied multimodal action model with grounded chain of thought and look-ahead spatial reasoning. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025. 2

[42] Andrew Szot, Alex Clegg, Eric Undersander, Erik Wijmans, Yili Zhao, John Turner, Noah Maestre, Mustafa Mukadam,

Devendra Chaplot, Oleksandr Maksymets, Aaron Gokaslan, Vladimir Vondrus, Sameer Dharur, Franziska Meier, Wojciech Galuba, Angel Chang, Zsolt Kira, Vladlen Koltun, Jitendra Malik, Manolis Savva, and Dhruv Batra. Habitat 2.0: Training home assistants to rearrange their habitat. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. 2, 3, 5, 1

[43] Andrew Szot, Max Schwarzer, Harsh Agrawal, Bogdan Mazoure, Rin Metcalf, Walter Talbott, Natalie Mackraz, R Devon Hjelm, and Alexander T Toshev. Large language models as generalizable policies for embodied tasks. In *International Conference on Learning Representations (ICLR)*, 2023. 1, 2, 3, 5, 6

[44] Andrew Szot, Bogdan Mazoure, Harsh Agrawal, R Devon Hjelm, Zsolt Kira, and Alexander Toshev. Grounding multimodal large language models in actions. *Advances in Neural Information Processing Systems*, 37:20198–20224, 2024. 1, 3, 5, 6

[45] Gemma Team, Aishwarya Kamath, Johan Ferret, Shreya Pathak, Nino Vieillard, Ramona Merhej, Sarah Perrin, Tatiana Matejovicova, Alexandre Ramé, Morgane Rivière, Louis Rouillard, Thomas Mesnard, Geoffrey Cideron, Jean bastien Grill, Sabela Ramos, Edouard Yvinec, Michelle Casbon, Etienne Pot, Ivo Penchev, Gaël Liu, Francesco Visin, Kathleen Kenealy, Lucas Beyer, Xiaohai Zhai, Anton Tsitsulin, Robert Busa-Fekete, Alex Feng, Noveen Sachdeva, Benjamin Coleman, Yi Gao, Basil Mustafa, Iain Barr, Emilio Parisotto, David Tian, Matan Eyal, Colin Cherry, Jan-Thorsten Peter, Danila Sinopalnikov, Surya Bhupatiraju, Rishabh Agarwal, Mehran Kazemi, Dan Malkin, Ravin Kumar, David Vilar, Idan Brusilovsky, Jiaming Luo, Andreas Steiner, Abe Friesen, Abhanshu Sharma, Abheesht Sharma, Adi Mayrav Gilady, Adrian Goedeckemeyer, Alaa Saade, Alex Feng, Alexander Kolesnikov, Alexei Bendebury, Alvin Abdagic, Amit Vadi, András György, André Susano Pinto, Anil Das, Ankur Bapna, Antoine Miech, Antoine Yang, Antonia Paterson, Ashish Shenoy, Ayan Chakrabarti, Bilal Piot, Bo Wu, Bobak Shahriari, Bryce Petrini, Charlie Chen, Charline Le Lan, Christopher A. Choquette-Choo, CJ Carey, Cormac Brick, Daniel Deutsch, Danielle Eisenbud, Dee Cattle, Derek Cheng, Dimitris Paparas, Divyashree Shivakumar Sreepathihalli, Doug Reid, Dustin Tran, Dustin Zelle, Eric Noland, Erwin Huizenga, Eugene Kharitonov, Frederick Liu, Gagik Amirkhanyan, Glenn Cameron, Hadi Hashemi, Hanna Klimczak-Plucińska, Harman Singh, Harsh Mehta, Harshal Tushar Lehri, Hussein Hazimeh, Ian Ballantyne, Idan Szpektor, Ivan Nardini, Jean Pouget-Abadie, Jetha Chan, Joe Stanton, John Wieting, Jonathan Lai, Jordi Orbay, Joseph Fernandez, Josh Newlan, Ju yeong Ji, Jyotinder Singh, Kat Black, Kathy Yu, Kevin Hui, Kiran Vodrahalli, Klaus Greff, Linhai Qiu, Marcella Valentine, Marina Coelho, Marvin Ritter, Matt Hoffman, Matthew Watson, Mayank Chaturvedi, Michael Moynihan, Min Ma, Nabila Babar, Natasha Noy, Nathan Byrd, Nick Roy, Nikola Momchev, Nilay Chauhan, Noveen Sachdeva, Oskar Bunyan, Pankil Botarda, Paul Caron, Paul Kishan Rubenstein, Phil Culliton, Philipp Schmid, Pier Giuseppe Sessa, Pingmei Xu, Piotr Stanczyk, Pouya Tafti, Rakesh Shivanna, Renjie Wu, Renke Pan, Reza Rokni, Rob Willoughby, Rohith Vallu, Ryan Mullins, Sammy Jerome, Sara Smoot, Sertan Girgin, Shariq Iqbal, Shashir Reddy, Shruti Sheth, Siim Põder, Sijal Bhatnagar, Sindhu Raghuram Panyam, Sivan Eiger, Susan Zhang, Tianqi Liu, Trevor Yacovone, Tyler Liechty, Uday Kalra, Utku Evci, Vedant Misra, Vincent Roseberry, Vlad Feinberg, Vlad Kolesnikov, Woohyun Han, Woosuk Kwon, Xi Chen, Yinlam Chow, Yuvein Zhu, Zichuan Wei, Zoltan Egyed, Victor Cotruta, Minh Giang, Phoebe Kirk, Anand Rao, Kat Black, Nabila Babar, Jessica Lo, Erica Moreira, Luiz Gustavo Martins, Omar Sanseviero, Lucas Gonzalez, Zach Gleicher, Tris Warkentin, Vahab Mirrokni, Evan Senter, Eli Collins, Joelle Barral, Zoubin Ghahramani, Raia Hadsell, Yossi Matias, D. Sculley, Slav Petrov, Noah Fiedel, Noam Shazeer, Oriol Vinyals, Jeff Dean, Demis Hassabis, Koray Kavukcuoglu, Clement Farabet, Elena Buchatskaya, Jean-Baptiste Alayrac, Rohan Anil, Dmitry, Lepikhin, Sebastian Borgeaud, Olivier Bachem, Armand Joulin, Alek Andreev, Cassidy Hardin, Robert Dadashi, and Léonard Hussenot. Gemma 3 technical report, 2025. 6

[46] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, 2022. 2, 6

[47] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. 2, 4

[48] Xue Yan, Yan Song, Xidong Feng, Mengyue Yang, Haifeng Zhang, Haitham Bou Ammar, and Jun Wang. Efficient reinforcement learning with large language model priors. In *International Conference on Learning Representations (ICLR)*, 2025. 2

[49] Rui Yang, Hanyang Chen, Junyu Zhang, Mark Zhao, Cheng Qian, Kangrui Wang, Qineng Wang, Teja Venkat Koripella, Marziyeh Movahedi, Manling Li, et al. Embodiedbench: Comprehensive benchmarking multi-modal large language models for vision-driven embodied agents. In *International Conference on Machine Learning (ICML)*, 2025. 1, 2, 3, 5, 6, 7, 8

[50] Yijun Yang, Tianyi Zhou, Kanxue Li, Dapeng Tao, Lusong Li, Li Shen, Xiaodong He, Jing Jiang, and Yuhui Shi. Embodied multi-modal agent trained by an llm from a parallel textworld. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 26275–26285, 2024. 1

[51] Fei Yu, Anningzhe Gao, and Benyou Wang. Ovm, outcome-supervised value models for planning in mathematical reasoning. In *Findings of the Association for Computational Linguistics: NAACL 2024*, pages 858–875, 2024. 2, 3

[52] Michał Zawalski, William Chen, Karl Pertsch, Oier Mees, Chelsea Finn, and Sergey Levine. Robotic control via embodied chain-of-thought reasoning. In *8th Annual Conference on Robot Learning*, 2024. 2, 4, 3

[53] Simon Zhai, Hao Bai, Zipeng Lin, Jiayi Pan, Peter Tong, Yifei Zhou, Alane Suhr, Saining Xie, Yann LeCun, Yi Ma, et al. Fine-tuning large vision-language models as decision-making agents via reinforcement learning. *Advances in neural*

information processing systems, 37:110935–110971, 2024. 1, 2

[54] Lunjun Zhang, Arian Hosseini, Hritik Bansal, Mehran Kazemi, Aviral Kumar, and Rishabh Agarwal. Generative verifiers: Reward modeling as next-token prediction. In *International Conference on Learning Representations (ICLR)*, 2025. 2, 3, 4

[55] Yaowei Zheng, Richong Zhang, Junhao Zhang, Yanhan Ye, Zheyan Luo, Zhangchi Feng, and Yongqiang Ma. Llamafactory: Unified efficient fine-tuning of 100+ language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, Bangkok, Thailand, 2024. Association for Computational Linguistics. 2

[56] Lizheng Zu, Lin Lin, Song Fu, Na Zhao, and Pan Zhou. Collaborative tree search for enhancing embodied multi-agent collaboration. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 29513–29522, 2025. 2

Think Twice, Act Once: Verifier-Guided Action Selection For Embodied Agents

Supplementary Material

First, we provide additional details about the benchmarks (App 8) and the training setup (App 9). Then, we provide the prompts used for synthetic data generation in App 10. Further, qualitative examples of synthetic mistakes and verifications are available in App 11. Finally, we provide some qualitative examples of the outputs generated by our verifiers at test time in App 12.

8. Details about Benchmarks

8.1. LangR [43]

The LangR benchmark [43], built on the Habitat 2.0 [42] simulator, is designed to evaluate the generalization capability of embodied agents in household rearrangement scenarios. Agents receive high-level instructions and must execute tasks that involve manipulating objects (pick, open, place), searching for target items, and performing simple forms of logical reasoning such as conditional operations. The allowed actions are: `navigate(receptacle)`, `pick(object)`, `place(object)`, `open(receptacle)`, `close(receptacle)`.

The actions are executed only if certain preconditions are satisfied. For example, an object can be picked only if it is within reach. The object categories used are: ball, clamp, hammer, screwdriver, padlock, scissors, block, drill, spatula, knife, spoon, plate, sponge, cleanser, plum, pear, peach, apple, lemon, can, box, banana, strawberry, lego, rubriks cube, book, bowl, cup, fork. The maximum number of steps for an episode is 32.

The benchmark provides a suite of training tasks along with a separate set of held-out test tasks. The test split is constructed to examine multiple aspects of generalization, including previously unseen environments and novel instruction formulations.

Two main forms of generalization are emphasized. The first is **paraphrastic robustness**, where the agent must interpret varied rephrasings of an instruction that shares the same underlying goal. The second is **behavioral generalization**, which requires the agent to handle new types of reasoning that do not appear in the training distribution. For example, during training the agent may learn to locate a specified number of object instances, while the multiple rearrangements task in the test set requires discovering all instances of a category without being told the exact count. We describe the tasks below.

Paraphrastic Robustness

- **Instruction Rephrasing:** Same underlying goal expressed using a different wording than in training.
- **Referring Expressions:** Objects are mentioned through descriptive or visual attributes rather than their canonical names (e.g., a banana described as a curved yellow fruit).
- **Context:** Objects are referred to within a situational or contextual description (e.g., a ball described as a sports object).
- **Irrelevant Instruction Text:** Additional text is included that does not affect the task but may distract the agent.

Behavioral Generalization

- **Multiple Rearrangements:** Requires rearranging three objects, although training tasks involve only two.
- **Novel Objects:** Introduces new combinations of instructions and object categories that never co-occur in training.
- **Multiple Objects:** Requires manipulating all instances of an object category. The agent must search for and detect every instance, a concept not present in training.
- **Conditional Instructions:** Task outcome depends on whether a specified condition holds (e.g., if the fridge is open, move the apple to it; otherwise move the orange, and only the required object).

The benchmark also includes a spatial reasoning task that uses instructions such as "place the object to the right of the black table." The action space available to the agent does not provide primitives for lateral (left, right, forward, back) movement, which prevents the agent from acquiring meaningful knowledge of the scene layout. As a result, the task is not compatible with the defined action space, so we exclude it from our evaluation.

8.2. ALFRED [33]

The ALFRED [33] benchmark is built on top of the AI2THOR simulator [16]. In this work, we use the EB-ALFRED implementation from [49], which restructures the original tasks into categories designed to probe different aspects of out-of-distribution generalization. The benchmark includes seven task types: *Pick & Place*, *Stack & Place*, *Pick Two & Place*, *Clean & Place*, *Heat & Place*, *Cool & Place*, and *Examine in Light*. Agents operate using eight possible actions: `pick up`, `open`, `close`, `turn on`, `turn off`, `slice`, `put down`, and `find`.

EB-ALFRED tasks are grouped into six subsets, each targeting a distinct skill or reasoning capability:

- **Base:** Evaluates core task solving abilities needed to plan and execute low to medium complexity action sequences.
- **Common Sense:** Measures the use of indirect object references grounded in everyday knowledge (for example, describing a refrigerator as "a receptacle that can keep food fresh for several days") and tests the agent's ability to apply such knowledge during instruction following.

- **Complex Instruction:** Contains longer contexts with both relevant and irrelevant details, assessing an agent's ability to extract the intended instruction.
- **Spatial Awareness:** Refers to objects through spatial relations with other items, testing spatial grounding and relational reasoning.
- **Visual Appearance:** Requires identifying objects based on visual attributes such as color or shape.
- **Long Horizon:** Includes tasks requiring extended action sequences, typically more than 15 steps in EB-ALFRED.

To construct the benchmark, [49] used the *valid seen* split of ALFRED. A set of 50 tasks with fewer than 15 steps was first selected, from which the common sense and complex instruction subsets were derived. Another 50 tasks with more than 15 steps formed the long horizon subset. Instances for the visual appearance and spatial awareness subsets were chosen directly from ALFRED based on language references to color, shape, or spatial relations. In total, EB-ALFRED contains 300 test instances, uniformly distributed across the six subsets (50 per subset).

9. Training Details

We train both the policy and verifier using the `LLaMAFactory` framework [55]. Full finetuning is applied while keeping the vision encoder and projection module fixed. All training runs are conducted on 8×NVIDIA L40 GPUs. Training data is formatted as multi turn dialogues using the `sharegpt` format provided by `LLaMAFactory`.

The inputs to the policy and verifier consist of prior images and actions rather than earlier chains of thought. To generate data compatible with this interface, each trajectory from the dataset is decomposed into a set of sub conversations, one for each action step. Consider an original trajectory of the form

$$I,\ o_1,\ (c_1, a_1),\ o_2,\ (c_2, a_2),\ \ldots$$

where I is the instruction, o_i are observations, c_i are chains of thought, and a_i are actions. The ith sub conversation contains the instruction along with all observations and executed actions up to step i:

$$I,\ o_1,\ a_1,\ o_2,\ a_2,\ \ldots,\ o_i,\ (c_i, a_i).$$

All chains of thought except the final one, c_i, are removed. During training we compute loss only on the last assistant message of each sub conversation, implemented by setting `mask_history` to `True`. Hyperparameters are provided in Table 6.

Table 6. Hyperparameters used for training

Hyperparameter	Value
Number of GPUs	8
per_device_train_batch_size	2
gradient_accumulation_steps	4
effective batch size	8 * 2 * 4 = 64
learning_rate	1e-5
num_train_epochs	3
warmup_ratio	0.1
bf16	True
lr_scheduler_type	cosine

10. Prompts

10.1. Prompt for CoT data generation

Prompt to Generate Synthetic Chain-of-Thought (adapted and modified from [52])

```
You're an expert reinforcement learning researcher. You've trained a policy for controlling a robot with an arm to move around and perform tasks in a household environment.
The robot successfully completed a task specified by the instruction: "{instruction}". For that purpose, the
robot executed a sequence of actions. Consecutive moves that were executed are the following:

insert original trajectory with instruction, images, actions


The attached images show what the robot was seeing at a given time step (corresponding to every <image> tag in the trajectory). You can use the images to understand what the robot was seeing, where it was, what it was holding, whether its actions were successful or not, etc.


## General Information and Instructions

1. possible actions are: pick(object), place_on_recep(receptacle), navigate(receptacle), open_fridge(), close_fridge(), open_cabinet(cabinet).
2. Possible objects are: ball, clamp, hammer, screwdriver, padlock, scissors, block, drill, spatula, knife, spoon, plate, sponse, cleanser, plum, pear, peach, apple, lemon, can, box, banana, strawberry, lego, rubrik's cube, book, bowl, cup, mug, orange, lid, toy airplane, wrench.
3. When exploring, select from possible receptacles: [cabinet, drawer 7, cabinet drawer 6, fridge, chair, black table, brown table, TV stand, sink, right counter, left counter]

## Your objective

I want you to annotate the given trajectory with reasoning. That is, for each step, I need to know not only which action should be chosen,
but importantly what reasoning justifies that action choice. I want you to be descriptive and include all the relevant information available.
The reasoning should include the task to complete, the remaining high-level steps, the high-level movements that should be executed and why they are required and any other relevant justification.


### Begin by describing the task

Start by giving an overview of the task. Make it more comprehensive than the simple instruction. Include the activity,
the objects the robot interacts with, and their relative locations in the environment. Then, describe
the high-level movements that were most likely executed, based on the task that was completed and the
primitive movements that were executed. Also, for each high-level movement write a justification for why it should be executed.
```

```
Write an answer for this part using markdown and natural language. Be descriptive and highlight all the relevant details, but ensure that your description is consistent with the trajectory that was  executed, specified above.

### List the reasonings for each step

Finally, for each step describe the reasoning that allows to determine the correct action. For each step describe the remaining part of the objective,
the current progress, the objects that are still relevant for determining the plan, and the plan for the next steps, based on the available features. Start the reasoning from a high level and gradually add finer features. I need you to be descriptive and very precise. Ensure that the reasoning is consistent with the task and the executed trajectory.
Write the answer for this part as a Python-executable dictionary. For every step in the initial trajectory there should be exactly one separate item of the form <step id>:<reasoning>.
Do not group the answers. The final dictionary should have exactly the same set of integer keys as the assistant messages in the original_trajectory above.
The reasoning should be a single string that describes the reasoning in natural language and includes all the required features.

Each reasoning string should have the following form:
- Describe the full task that remains to be completed (but only describe what remains), and place it inside a tag <task>.
- Describe the complete high-level plan for completing the remaining task (the list of remaining high-level steps), and place it inside a tag <plan>.
- Describe why the chosen high-level step should be executed now, which features of the current environment influence that decision, and how it should be done. Place it within a tag <subtask_reason>. First provide appropriate reasoning, and then the name of the action chosen (and not, "I choose action XYZ because ..."; rather, "I need to do XYZ, hence I should choose this action")
- Describe the high-level step that should be executed now (chosen from the list of high-level steps), and place it inside a tag <subtask>.



## Reminders

Keep in mind that the robot might not know where a certain object is, and it may have to move around the environment to search for it. Searching might be one of your subtasks.

Also, it's possible that the actions chosen by the robot are sometimes suboptimal, you should take that into account.
Your output reasoning chains should be from the perspective of the assistant, about how it should reason before picking the next action.
Note that the small pebble like thing near the gripper is part of the gripper itself, not a separate item.
The reasoning should be such that first it states why a certain action should be chosen, and then specify that action ("I need to find XYZ, hence, I should do action A" and not "I should do action A. It will help me find XYZ").
Sometimes, the robot would select non-sensical actions, you don't necessarily have to justify such actions (for example by hallucinating objects that don't exist).
You can use information from all time steps to develop your plan, but your final output should be such that the output at time t doesn't assume information about the future
```

```
     (because the robot will think and act one step at a time).
Also, in the original trajectory, after every assistant action, there is a flag telling
    whether that action was successful or not. You can use this to inform your reasoning
     chain output. However, note that during deployment, the agent will not have access
    to this information directly. Instead, it must use its visual input to decide
    whether the action succeeded. Hence, in your output reasoning chains, when you say
    that a previous action was successful/failed, you must say that this is based on the
     image (you can use the ground truth success status, but don't say it in the output)
    .
In the output reasoning trajectory, it is okay if the robot changes the original plan/
    subtasks as it obtains more information about the environment, or after failures,
    similar to how an intelligent agent would adapt as it works toward solving a task.
    The robot should also be capable of recovering from previous errors. Such changes
    should be clearly stated in the reasoning string.

## Task summary

Here is a breakdown of what needs to be done:

- Describe the task.
- Describe the high-level movements that were executed, based on the completed task and
    the listed features.
- Describe the plan for the solution that allowed the robot to complete the task
    successfully.
- For each step on the trajectory, describe the reasoning that leads to determining the
    correct action. The reasoning should be descriptive and precise. You should provide
    exactly one reasoning
string for each ASSISTANT step on the trajectory specified by original_trajectory.
- Just return the dictionary directly, like:
'''python
{{
  ...
}}
'''
- At the very end of the response, write a single label FINISHED to indicate that the
    answer is complete.
```

10.2. Prompts for Synthetic Failed Trajectory Generation

Prompt to Generate Synthetic Mistakes and Verifications

```
Given the above conversation, your job is to create a new trajectory where the agent
    makes a mistake. Don't change the instruction, just imagine some mistakes and insert
     them into the new trajectory.

Your mistakes should be realistic and plausible, i.e., something that the agent is
    actually likely to do, and not a random mistake. For example:
- The agent misunderstands the instruction. it interacts with the incorrect object, or
    with incorrect receptacles, or doesn't perform the expected operations on the
    objects/receptacles. Here, try to make the "incorrect" actions still somewhat close
    to the correct actions, e.g., replacing apple with orange instead of remote, or
    replacing turn on with turn another related action instead of put down the object.
- The agent doesn't complete a precondition for the next action, e.g., it does not 'find
```

```
    ' the object before interacting with it, or doesn't 'open' the microwave or cabinet
    before putting the object inside. this one is quite important -- basically, think of
     ways in which the agent might be incompetent: even if it understood the task
    correctly, it might fail to perform it properly

- The agent messes up the order in which actions should be done. for example, if it has
    to clean something, it puts the object in the sink, takes it out, and then turns on
    the faucet

- The agent gets confused between different instances of the same object

- The agent completes only one part of the instruction and not the full instruction

- The agent doesn't do all the actions required for a subgoal, for example, it cleans an
     object in the sink, but does not pick it up before putting it somewhere else.


Keep in mind that the original trajectory is successful, so if you insert a mistake
    there, your action should be different from the action given in the original
    trajectory, because otherwise that action would be correct.

The output should also be in a similar json format. additionally, for every action in
    the output, add a judgement on whether the action is correct or incorrect. first,
    provide detailed reasoning, and then the verdict ("action_is_correct: yes or no").
    the verdict and judgement should be a single string and should go in a separate key
    in the dict called 'verification'. also, it's possible that some actions are correct
     and others are incorrect in the same conversation.

Also, per conversation, add a mistake_description field, giving a summary of what the
    mistake was.
Make sure that in the output, before every assistant message there is a user message.
for example:

<new_trajectory>
mistake_summary: <insert mistake summary>

{
"content": "...",
"role": "user",
},
{
"content": "...",
"role": "assistant",
"is_action_successful": true,
"verification": "<insert reasoning here>\n
is_action_correct: X" where X is yes or no,
},


**IMPORTANT**:
- Judge every action on whether it advances the task or not. so it is possible that the
    reasoning for the action is incorrect, but it still advances the task, in that case,
     the action is correct.
- The mistakes should start from the task interpretation, planning, and subtask
    reasoning itself, and not like it understands the task but the subtask/action are
```

```
    not aligned with the reasoning.
- Keep in mind that the output conversation is from the perspective of the agent, and
    the agent doesn't know it's making a mistake. Make sure that in the output the agent
     should not seem to know that its plans and actions are not aligned with the
    instruction. The agent should think its doing the right thing, so it doesn't even
    know that it misunderstood the instruction. so there should not be anything like "
    doing X even though the instruction said Y" or "mistakenly doing X".
- Try to make a 'realistic' mistake - go for an object that is somewhat related to the
    actual object mentioned in the instruction.
- In the verification, try to explain your reasoning in as much detail as possible,
    writing out which object was targeted and which should have been targeted instead.
- Try to generate diverse types of mistakes. You don't have to restrict yourself to the
    mistakes mentioned above, feel free to come up with other realistic failure modes.
- Keep in mind that the original trajectory is successful, so if you insert a mistake
    there, your action should be different from the action given in the original
    trajectory, because otherwise that action would be correct.
- Simply using a different object/receptacle/action might not be considered a mistake.
    For example, if a lamp is required, both floorlamp and desklamp are correct.
- Make sure that every assistant message has all the relevant tags (<task>, <plan>, <
    subtask>, etc., and especially the <action> tag)
- In the verification, first try to reason about what the instruction specifies and the
    action taken by the agent, how the two are related, what the agent should have done,
     and describe in detail why the action is incorrect.
- Every verification will be used independently of each other, so every verification
    should be complete in itself (shouldn't assume existence of previous verifications).

Double check for these important points before writing out the output conversation. It
    is crucial that these instructions are followed. So check rigorously and strictly
    your output, and revise them to follow these instructions if necessary.
```

11. Examples of Synthetic Incorrect Actions

Here, we show some examples of trajectories with synthetic mistakes generated by the LLM to train the verifier.

11.1. LangR

Example 1 (LangR): The agent erroneously picks up the bowl even though the instruction specifies a cleaner.

```
Instruction: Shift a cleanser and place it into the specified black table.


action 1: "<task>Move the bowl over to the black table</task>\n<plan>1. Go to right
    kitchen counter where the bowl usually sits.\n2. Pick up the bowl.\n3. Navigate to
    the black table.\n4. Place the bowl on the black table.</plan>\n<subtask>Navigate to
     the right kitchen counter.</subtask>\n<subtask_reason>The bowl is often left on the
     right side of the kitchen counter, so I will head there first.</subtask_reason>\n<
    action>('nav(receptacle_aabb_counter_right_kitchen_counter)', 10)</action>"

verification: "The instruction asks for relocating a cleanser, but the assistant plans
    to move a bowl instead. Nevertheless, navigating to a kitchen counter could
    logically be a first step for finding a cleanser as well, so this motion does not
    directly violate task progress.\naction_is_correct: yes",
```

```
=====

action 2: "<task>Move the bowl over to the black table</task>\n<plan>[ Pick bowl --> Go
    to black table --> Place bowl ]</plan>\n<subtask>Pick up the bowl.</subtask>\n<
    subtask_reason>I can see the bowl on the counter and will grab it.</subtask_reason>\
    n<action>('pick_bowl(robot_0)', 42)</action>",

verification: "The picked object is a bowl, not the requested cleanser, so this action
    does not advance the actual task.\naction_is_correct: no"

<following actions omitted for clarity>
```

Example 2 (LangR): The agent erroneously navigates to Table 1 instead of the TV stand after picking up the object

```
Instruction: Place both a plum and a plate onto the TV stand.

action 1: "<task>Place a plum and a plate on the TV stand.</task> <plan>['navigate to
    sink counter', 'pick plum', 'navigate toward TV-stand area', 'place plum', 'search
    for plate', 'place plate']</plan> <subtask>Navigate to sink counter</subtask> <
    subtask_reason>The plum is normally kept on the kitchen counter near the sink, so
    the first step is to move there.</subtask_reason> <action>('nav(
    receptacle_aabb_sink_kitchen_counter)', 9)</action>",

verification: "The instruction requires obtaining the plum first. Moving from the start
    position to the sink counter is the expected way to reach the plum, so this movement
     advances the task.\naction_is_correct: yes",

=========

action 2: "<task>Place a plum and a plate on the TV stand.</task> <plan>['pick plum', '
    navigate toward TV-stand area', 'place plum', 'search for plate', 'place plate']</
    plan> <subtask>Pick plum</subtask> <subtask_reason>The plum is visible on the
    counter and now within reach, so it should be picked up.</subtask_reason> <action>('
    pick_plum(robot_0)', 30)</action>",

verification: "Grasping the plum is necessary before transporting it anywhere. This
    action clearly moves the task forward.\naction_is_correct: yes"

==========

action 3: "<task>Deliver the plum.</task> <plan>['navigate to TV-stand area', 'place
    plum', 'search for plate', 'place plate']</plan> <subtask>Navigate to TV-stand area
    </subtask> <subtask_reason>The low wooden table in front of the sofa is identified
    as the surface directly in front of the television, so I should move there to drop
    the plum.</subtask_reason><action>('nav(
    receptacle_aabb_Tbl1_Top1_frl_apartment_table_01)', 6)</action>",

verification: "The instruction specifies the TV stand, not Table 1. Navigating to Table
    1 therefore heads to the wrong receptacle and does not directly advance the true
    goal.\naction_is_correct: no",

<following actions omited for clarity>
```

11.2. ALFRED

Example 1 (ALFRED): The agent erroneously picks up the baseball instead of the tennis racket, and turns the lamp off instead of on.

```
Instruction: Grab a tennis racket between beds, turn on the lamp

action 1: "<task>I need to: (1) collect the elongated sporting object I can see between
    the two beds, and (2) operate the nearby lamp.</task>\n<plan>1) Focus on the bat-
    shaped item between the beds and secure it. 2) Locate the desk lamp and toggle its
    power.</plan>\n<subtask>Look for the sporting item I believe to be a BaseballBat.</
    subtask>\n<subtask_reason>The long, thin handle and rounded head suggest a bat; I
    will therefore issue a search for a BaseballBat.</subtask_reason>\n<action>find a
    BaseballBat</action>",

verification: "The instruction requires the agent to locate a TennisRacket, but the
    agent mistakenly assumes the object is a BaseballBat and executes a search for that
    instead.  Although the 'find' command itself executes, it advances the wrong goal.\n
     is_action_correct: no",


============


action 2: "<task>The sporting item has been detected.  I will now secure it.</task>\n<
    plan>Approach the object and grasp it.</plan>\n<subtask>Pick up the BaseballBat.</
    subtask>\n<subtask_reason>With the bat in view, gripping it by the handle completes
    the first half of the instruction.</subtask_reason>\n<action>pick up the BaseballBat
    </action>",

verification: "No BaseballBat is present; the elongated object is actually a
    TennisRacket.  Because the target object does not exist, the simulator would return
    failure.  Attempting this pick-up neither fulfils nor advances the stated task.\n
    is_action_correct: no",


============


action 3: "<task>With the sporting item handled, I will now take care of the lamp.</task
    >\n<plan>Find the DeskLamp, then toggle its power.</plan>\n<subtask>Locate the
    DeskLamp.</subtask>\n<subtask_reason>The lamp is required next; I must bring it into
     view before interacting.</subtask_reason>\n<action>find a DeskLamp</action>",

verification: "Searching for the lamp is sensible even though the racket hasn't been
    obtained yet.  The action does move the agent toward one component of the goal, so
    the step is reasonable.\n is_action_correct: yes"


============


action 4: "<task>The lamp is now in reach.  I will toggle it.</task>\n<plan>Interact
    with the lamp's switch.</plan>\n<subtask>Switch the DeskLamp off.</subtask>\n<
    subtask_reason>The lamp appears lit in my view; turning it off will leave it in the
    desired off state.</subtask_reason>\n<action>turn off the DeskLamp</action>",
```

```
verification: "The instruction explicitly says to turn the lamp ON, not OFF.  The agent
    does the opposite, so even though the environment executes the switch action this
    contradicts the task requirement.\n is_action_correct: no",
```

Example 2 (ALFRED): The agent erroneously places the wine bottle instead of the vase on the table.

```
Instruction: Put a glass vase on a table


action 1: "<task>Instruction outstanding: place the vase on a table.</task>\n<plan>1)
    Locate a suitable decorative vessel; 2) Pick it up; 3) Bring it to a table; 4) Set
    it down.</plan>\n<subtask>Begin by locating a nice glass vessel (going to look for a
     WineBottle that will serve as the vase).</subtask>\n<subtask_reason>The shelves
    often hold tall glass containers; a wine bottle can double as a vase.</
    subtask_reason>\n<action>find a WineBottle</action>",

verification: "The instruction explicitly says to put a glass vase (object class
    Glassbottle/Vase) on a table. The assistant chooses to search for a WineBottle
    instead. Although both are made of glass, a wine bottle is not the requested vase,
    so this action does not directly progress toward satisfying the instruction.\
    naction_is_correct: no",


=============


action 2: "<task>Vessel found; next step is to secure it.</task>\n<plan>Pick the bottle
    up, then head to a table.</plan>\n<subtask>Grasp the WineBottle.</subtask>\n<
    subtask_reason>The object is within reach and I need to carry it to the table.</
    subtask_reason>\n<action>pick up the WineBottle</action>",

verification: "Again, the agent manipulates a WineBottle instead of the required glass
    vase. Picking it up succeeds mechanically, but the chosen object is incorrect for
    the task.\naction_is_correct: no",


=============


action 3: "<task>Object secured; now need a table.</task>\n<plan>Navigate to a dining
    table in the room.</plan>\n<subtask>Find a DiningTable.</subtask>\n<subtask_reason>
    Any table surface will work as a display spot.</subtask_reason>\n<action>find a
    DiningTable</action>",

verification: "Navigating toward a table is generally required, but because the agent is
     holding the wrong object, this search does not really bring it closer to completing
     the instructed task. Therefore the action, while plausible, does not advance the
    true goal.\naction_is_correct: no",


=============
```

```
27
28 action 4: "<task>Table located; time to place the decorative bottle.</task>\n<plan>
       Release the bottle onto the table surface.</plan>\n<subtask>Put the carried bottle
       down on the table.</subtask>\n<subtask_reason>The receptacle is in reach and I no
       longer need to transport the item.</subtask_reason>\n<action>put down the object in
       hand</action>",
29
30 verification: "The assistant places the WineBottle on the table instead of a glass vase.
        The receptacle is correct (a table), but the object does not match the instruction.
        Consequently, the high-level task remains unsatisfied.\naction_is_correct: no",
```

12. Additional Qualitative Examples of Verification During Inference

12.1. LangR

Instruction: Procure a purple fruit and deliver it to the left counter.

Sampled Action: <task>Find and pick up the purple fruit and then deliver it to the left counter.</task> <plan>[Search TV stand, search sofa, search table2, pick fruit, navigate to left counter, place fruit]</plan> <subtask_reason>At the TV stand, I should try a pick to see if the fruit is here, hence pick(object).</subtask_reason> <subtask>Attempt to pick the fruit at the TV stand.</subtask> <action>('pick_strawberry(robot_0)', 38)</action>

Verification CoT: The assistant tries to pick a strawberry instead of the requested purple fruit. This does not help fulfil the instruction and therefore does not advance the task. **action_is_correct: no**

Figure 7. Verification example on the LangR benchmark. Here, the task is to move a "purple fruit", but the agent tries to pick up a strawberry. The verifier correctly identifies that the strawberry is not the right object, classifying the action as incorrect.

Instruction: Move the rubriks cube to the brown table, the lego to the brown table, and the bowl to the brown table.

Sampled Action: <task>Remaining: locate and pick the bowl (it was not found at the TV stand).</task> <plan>[Return to table1 to search for bowl, then cabinet area, pick bowl, navigate to brown table, place bowl]</plan> <subtask_reason>I am at the brown table and need to check if the bowl is here, hence I should attempt a place action to see if it is present</subtask_reason> <subtask>attempt to place bowl on brown table</subtask> <action>('place(receptacle_aabb_Tbl2_Top1_frl_apartment_table_02)', 51)</action>

Verification CoT: The robot is not holding any object (the previous pick attempt failed and no new object was picked). Attempting to place an item while not holding anything cannot succeed and does not progress the task. **action_is_correct: no**

Figure 8. Verification example on the LangR benchmark. Here, the agent erroneously attempts to place the bowl without successfully picking it up first. The verifier identifies this mistake, classifying the action as incorrect.

12.2. ALFRED

Instruction: Place washed leafy green vegetable in a receptacle that can keep it fresh for several days

Sampled Action: <task>The fridge is open and I (mistakenly) see another lettuce inside; I still intend to store the held lettuce.</task> <plan>[Optional] Decide whether to store this extra lettuce or switch back to my original intent, then ...</plan> <subtask_reason>I am uncertain of needing both wastebulbs; closing keeps the environment tidy while I reconsider.</subtask_reason> <subtask>Close the fridge door.</subtask> <action>close the Fridge</action>

Verification CoT: Closing the fridge at this moment is counter-productive because the lettuce needs to remain inside to stay fresh. Although syntactically valid, it moves the state away from the goal. **action_is_correct: no**

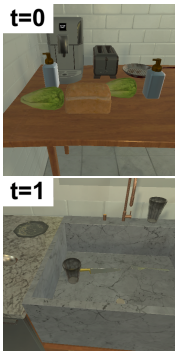
Figure 9. Verification example on the ALFRED benchmark. Here, the agent incorrectly attempts to close the fridge before putting the lettuce inside because it mistakenly detects a second lettuce in the fridge. The verifier identifies this mistake, classifying the action as incorrect.

Instruction: As you prepare for dinner, arrange an eating surface that will hold your food with a spoon in it on the kitchen table.

Sampled Action: <task>Entire task still outstanding: create a state where the dining-table has a container holding a spoon resting on it.</task> <plan>[1] locate a spoon-like utensil, [2] pick it up, [3] find a bowl or cup, [4] place utensil inside it, [5] carry the filled utensil to the dining-table, [6] set it down on the table.</plan> <subtask_reason>Before anything else I need a spoon; without knowing its position I cannot grasp or use it. Therefore I must visually search for an appropriate utensil first.</subtask_reason> <subtask>Locate a spoon-like utensil.</subtask> <action>find a Ladle</action>

Verification CoT: The user's instruction explicitly calls for a "spoon." The agent instead decides to search for a Ladle, which is a different utensil. Although both are eating utensils, a ladle is not the requested object. Thus the chosen action does not advance the stated goal directly. **action_is_correct: no**

Figure 10. Verification example on the ALFRED benchmark. Here, the task is to move a spoon, but the agent proposes to move a ladle, which is not in the scene. The verifier correctly identifies that the correct object is a spoon, classifying the action as incorrect.



Instruction: During a cooking session, keep a clean flipping tool on a wooden table.

Sampled Action: <task>Entire task still outstanding: locate a suitable flipping utensil, make sure it is clean, and finally leave it resting on the wooden dining table.</task> <plan>1) Find a knife. 2) Pick it up. 3) Go to a sink. 4) Wash the knife by turning water on then off. 5) Bring the clean knife to the wooden dining table and set it down there.</plan> <subtask_reason>The very first requirement is to know where a knife is; without visual contact I cannot interact with it or progress toward washing or placing. Therefore I need to perform a search action that will move my sensors through the scene until a knife is detected.</subtask_reason> <subtask>Find a knife</subtask> <action>find a Knife</action>

Verification CoT: The instruction asks for any kind of cleaning flipping tool (i.e., a knife). By performing the action "find a Knife" the agent begins the search for such an object, which is a reasonable first step toward accomplishing the goal. **action_is_correct: yes**

Figure 11. Verification example on the ALFRED benchmark. Here, the task is to move a spatula, but the agent proposes to move a knife. The verifier fails to identify this mistake and incorrectly classifies the action as correct.